GOVERNMENT POLYTECHNIC COLLEGE, PATNA 7

PYTHON PROGRAMMING

MICRO-PROJECT REPORT

STUDENT RESULT CALCULATOR

Submitted in partial fulfilment of the requirements for the subject

Python Programming

Department: Computer Science and Engineering (CSE)
Semester: 3rd Semester
Academic Session: 2026–2027

PROJECT TEAM

S. No.	Name	Roll Number
1	Krish Kumar	20/CSE/25
2	Alok Raj	21/CSE/25
3	Priya Kumari	22/CSE/25
4	Aditya Raj	23/CSE/25
5	Sonu Bhardwaj	24/CSE/25
6	Puja Kumari	27/CSE/25

Submitted to: _____

Project Guide / Faculty: _____

1. INTRODUCTION

The result of an examination is more than a collection of marks. It gives a student an idea of how well the topics have been understood and helps teachers communicate academic progress. Once marks are awarded, they must be totalled, converted into a percentage and interpreted according to the grading rules followed by the institution. Although these calculations are straightforward, repeating them for several subjects and students can become time-consuming.

The Student Result Calculator is a small application developed in Python to make this process easier. It accepts marks in five subjects—English, Mathematics, Computer Science, Physics and Chemistry—and uses the entered values to calculate the aggregate, percentage, grade and final pass/fail status. The program also checks the marks before using them, because a calculation is only useful when the data entered is valid.

The project has been selected because it connects programming concepts with a familiar academic task. A student learning Python can see how input statements collect information, variables hold values, arithmetic operators perform calculations and conditional statements make decisions. Instead of treating these concepts as separate examples, the project brings them together in one complete program.

The program is intended as an educational demonstration. It shows the basic logic behind result calculation, but it is not an official college result-management system. A real system may need student records, user accounts, secure storage, approval procedures and rules verified by the institution. The current project focuses on the calculation itself so that its working can be understood and explained clearly.

2. BACKGROUND AND NEED OF THE PROJECT

In a typical academic setting, a result has to be prepared after marks for individual subjects have been recorded. The marks are added to obtain a total, and the total is compared with the maximum possible marks to find the percentage. A grading rule is then applied, while the minimum passing requirement determines whether the student has passed. These steps are repeated for every result that needs to be prepared.

When the number of students is small, manual calculation may seem convenient. However, the same steps must be repeated carefully each time. A subject can be missed while adding marks, a total may be copied incorrectly, or a grade boundary may be applied inconsistently. Even a small arithmetic mistake can make the displayed result incorrect. A computer program can perform the same operations in the same order whenever it is given valid input.

The need for this project is therefore not simply to replace addition with a computer. It is to demonstrate a consistent process in which input is checked, calculations are performed using a defined formula and decisions follow written rules. The program should also give a clear message when an entry cannot be accepted, rather than quietly using an invalid value.

For a beginner, the project provides practice in breaking a problem into smaller tasks. The developer must decide what information is required, how the marks will be stored, which formula will be used and how the final grade will be chosen. These planning decisions are just as important as writing the code. Once the basic version works, it can be expanded gradually without changing the purpose of the original program.

3. PROBLEM STATEMENT

The problem addressed by this project is the repetitive calculation of a student's result from marks obtained in several subjects. A correct result requires more than finding a total. The program must also ensure that the entered marks are valid, calculate the percentage, apply the grade thresholds in the correct order and check the passing condition for every subject.

The application will accept five marks, each with a maximum value of 100. Therefore, the maximum aggregate is 500. A mark must be numeric and must fall between 0 and 100, inclusive. If a user enters letters, a negative number or a number greater than 100, the program should reject the entry and ask for a valid mark.

The program must apply a minimum passing mark of 33 in every subject. This condition is important because a high total does not necessarily mean that a student has passed all subjects. If even one subject has a mark below 33, the final status must be FAIL. Only after this check should a passing grade be assigned.

The required output is a compact result summary showing each subject's marks, the total out of 500, the percentage, the grade and the final status. The purpose is to make the calculation quick, repeatable and easy to verify. The grading thresholds used in this report are project rules for demonstration and should be replaced if an official institutional policy specifies different rules.

4. OBJECTIVES OF THE PROJECT

The main objective of the Student Result Calculator is to develop a simple Python program that performs basic result calculations automatically. The project is designed to be small enough for a student to understand, but complete enough to demonstrate how a real task can be converted into a working program.

The first objective is to collect marks for five clearly named subjects. The program should make it obvious which subject is being requested so that the user does not enter a value in the wrong place. Each value must be checked before it is accepted. Validating the input prevents values outside the permitted range from affecting the result.

The second objective is to calculate the aggregate and percentage accurately. Since each subject carries 100 marks, the maximum total is 500. The percentage is obtained by dividing the total by the maximum total and multiplying by 100. The program then uses the percentage and subject-wise passing condition to decide the grade and status.

Another objective is to demonstrate core Python features in a meaningful context. These include variables, lists, loops, input/output, type conversion, arithmetic expressions, conditional statements and exception handling. The project also aims to produce readable output and code that can be modified later. A future version could accept student details, process a class list or save the result in a file.

5. SCOPE AND APPLICATION

The current version is a console-based program for one student's result. It uses five fixed subjects and assumes that each subject has a maximum of 100 marks. The program validates the input, calculates the total and percentage, assigns a grade and displays the final status. This limited scope keeps the main logic straightforward and makes it appropriate for a micro-project.

The program can be used during practical classes to demonstrate how a small software solution is planned and implemented. It can also be used as a learning exercise for students who are beginning to understand loops, conditions and exception handling. By changing the marks entered, the user can observe how different inputs affect the total, grade and pass/fail decision.

The project does not currently store results after the program ends. It does not provide a login screen, student database, graphical interface, multiple examination terms or an official report-card format. It also does not verify whether the entered marks match a teacher's official record. These features are outside the scope of the basic version.

The project can be extended in several directions. A list of students could be processed in one run, results could be saved to a CSV file, and a database could be used to retrieve student information. A graphical interface could make data entry easier for non-technical users. Before any real deployment, the grading rules and data-handling procedures would need to be checked and approved by the institution.

6. REQUIREMENTS ANALYSIS

6.1 Functional requirements

The program must ask the user to enter marks for all five subjects. It must validate each entry, reject non-numeric values and prevent marks below 0 or above 100 from being accepted. After collecting valid marks, it must calculate the total and percentage, determine the grade according to the specified thresholds, check the pass condition and display the final result. If the entry is invalid, the program should provide a message that helps the user correct it.

6.2 Non-functional requirements

The application should be easy to run and understand. Its messages should be clear, and the calculations should be consistent for the same input. The source code should be readable enough for another student or teacher to follow. The program should not depend on external packages for its basic operation, which makes it easy to execute in a standard Python environment.

6.3 Hardware and software requirements

A basic laptop or desktop computer is sufficient. The program requires Python 3.x and a text editor or integrated development environment such as IDLE or Visual Studio Code. It can also be run from a terminal. Since the program uses built-in Python features, no additional library installation is required.

6.4 User requirements

The user should know the marks that need to be entered and should provide one mark at each prompt. If the program reports an invalid value, the user should enter a corrected value. The user should also understand that the grade thresholds in this demonstration are examples and may not match the official rules used by every institution.

7. INTRODUCTION TO PYTHON

Python is a high-level, general-purpose programming language. Its syntax is designed to be readable, and it is widely used for automation, web development, data analysis, scripting and education. Its clear structure makes it a useful choice for beginners who are learning how to express a problem as a sequence of instructions.

A Python program is commonly saved in a file with the extension .py. The Python interpreter executes the statements in that file. A program can be run from an editor, an integrated development environment or a terminal. When the code contains a syntax error, the interpreter reports the problem so that the programmer can correct it.

Python includes many built-in functions that are useful in this project. The input() function collects text from the user, while print() displays messages and results. Functions such as float() convert numeric text into a number, and round() can control the precision of a displayed value. The sum() function can add all the marks stored in a list.

The language also provides decision-making statements and loops. An if statement checks a condition and executes a block when the condition is true. The elif and else statements allow other cases to be handled. A for loop can visit each subject in a list, while a while loop can repeat an input prompt until a valid mark is provided. These features make it possible to write the result calculator without complicated external tools.

8. PYTHON CONCEPTS USED IN THE PROJECT

8.1 Variables and data types

A variable is a name used to refer to a value in a program. The calculator uses numeric values to represent marks, total and percentage. Text values are used for subject names, grade and status. Choosing meaningful variable names makes the code easier to read. For example, the name percentage is clearer than a short name such as p.

8.2 Input and output

The input() function receives information typed by the user. By default, the value returned by input() is text, so a conversion is required before the value can be used in arithmetic. The print() function displays the result. Labels such as 'Total' and 'Percentage' help the user understand the meaning of each value rather than seeing a list of unexplained numbers.

8.3 Arithmetic operators

The plus operator is used when adding marks. Division is used to compare the total with the maximum possible marks, and multiplication by 100 converts the ratio into a percentage. The formula used is percentage = total / maximum_total * 100. For example, a total of 400 out of 500 gives 80 percent.

8.4 Conditional statements

The grade is selected by comparing the percentage with thresholds. Conditions should be checked from the highest threshold downwards. If the program checks 80 before 90, a percentage of 92 could incorrectly receive grade A instead of A+. The pass/fail condition is checked separately because one failed subject must result in FAIL even if the aggregate is high.

8.5 Loops and exception handling

A loop avoids writing nearly identical input statements for every subject. The program can repeat the prompt until the entered value is valid. A try/except block handles a ValueError that may occur when the user enters letters instead of a number. Together, the loop and exception handling make the program more robust and user-friendly.

9. SYSTEM DESIGN AND WORKING PRINCIPLE

The program follows the familiar input-process-output model. In the input stage, the user enters the marks for each subject. During processing, the program validates each mark, stores the accepted values, calculates the aggregate and percentage, and applies the grade and passing rules. The output stage presents the result in a readable summary.

The order of operations matters. The program should not calculate the final result from incomplete or invalid data. Each mark is accepted only after it has been converted to a number and checked against the permitted range. Once all five marks have been collected, the program can safely calculate the total and percentage.

The subject-wise passing condition is checked before a passing grade is assigned. If any mark is below 33, the result is FAIL and the grade is F. Otherwise, the percentage is compared with the grade thresholds. This separation between pass status and percentage grade avoids the mistake of assuming that a high aggregate automatically means that every subject has been passed.

The system is deterministic, meaning that the same valid marks and the same rules produce the same output. No random values or external data are involved. The calculation can therefore be checked manually using a sample result, which is useful when demonstrating the project to a teacher.

10. ALGORITHM

An algorithm is a finite sequence of steps that describes how a problem will be solved. Preparing an algorithm before coding helps the programmer check that all required operations have been included and that the order of decisions is correct.

First, the program starts and defines the five subject names, the maximum mark for each subject and the minimum passing mark. It then asks the user to enter a mark for the first subject. The value is converted to a number. If conversion fails, the program displays an error message and asks again. If the value is outside the range 0 to 100, it is rejected and the prompt is repeated.

The same validation process is followed for every subject. After all five marks have been accepted, the program adds them to calculate the total. It divides the total by 500 and multiplies the result by 100 to obtain the percentage. Next, it checks whether any subject mark is less than 33. If that condition is true, the result is FAIL and the grade is F.

If every subject has at least 33 marks, the program assigns a grade according to the percentage: A+ for 90 or above, A for 80 or above, B for 70 or above, C for 60 or above, D for 45 or above, and E for a lower passing percentage. Finally, it displays all subject marks, the total, percentage, grade and status, and then ends.

11. FLOWCHART EXPLANATION

A flowchart is a visual representation of an algorithm. It uses standard shapes to show the type of operation being performed. An oval is commonly used for Start and End, a parallelogram represents input or output, a rectangle represents a processing step, and a diamond represents a decision. Arrows show the direction in which control moves through the program.

In this project, the flowchart begins with Start and moves to the input stage. The user enters a mark, and the program checks whether it is numeric and within the permitted range. If the value is invalid, the flow returns to the input step. If the value is valid, the program stores it and continues to the next subject until all five marks have been entered.

After the input stage, the flow moves to the calculation steps. The total is found by adding the marks, and the percentage is calculated using the maximum total. A decision diamond then checks whether any subject mark is below 33. The 'yes' branch leads to FAIL and grade F. The 'no' branch leads to the grade-selection steps, where the percentage is compared with the thresholds.

The final stage displays the result and reaches End. This flow makes the decision order visible and helps identify missing cases. In particular, the flowchart shows that subject-wise passing is checked before assigning any of the passing grades.

12. GRADING SYSTEM AND PASS CRITERIA

A grading system converts numerical performance into a smaller set of categories that are easier to interpret. The exact thresholds vary between institutions, courses and examination systems. For this micro-project, a simple demonstration policy is used so that the grading logic can be tested and explained.

Each subject carries a maximum of 100 marks, and the student must score at least 33 in every subject to pass. With five subjects, the maximum aggregate is 500. The percentage is calculated by dividing the total marks by 500 and multiplying by 100. The result is displayed to two decimal places for readability.

The grade thresholds are as follows: A+ is awarded for a percentage of 90 or above; A for 80 to below 90; B for 70 to below 80; C for 60 to below 70; D for 45 to below 60; and E for a percentage below 45 when all subjects have been passed. If any subject mark is below 33, the result is FAIL and grade F, regardless of the aggregate percentage.

For example, marks of 80, 90, 85, 75 and 70 give a total of 400. Dividing 400 by 500 and multiplying by 100 gives 80 percent, so the student receives grade A and PASS. By contrast, a student with 30 in one subject fails the subject-wise condition even if the other marks are high. The thresholds in this report are for the project demonstration only and should not be represented as the official college grading policy.

13. PROGRAM IMPLEMENTATION

Program implementation is the stage where the planned algorithm is translated into executable Python statements. The implementation begins by storing the subject names in a list. A list is useful because it keeps related items together and allows the program to process them in a loop. The marks are collected into a second list in the same order as the subject names.

For each subject, the program asks for a mark inside a while loop. The loop continues until the value is valid. The conversion to float is placed inside a try block so that a non-numeric entry can be handled without terminating the program. A range check then ensures that the accepted mark is between 0 and 100.

After input collection, the sum() function calculates the total. The maximum total is obtained from the number of subjects multiplied by the maximum mark per subject. This approach is slightly more flexible than typing 500 directly into the formula, because the maximum can adjust if the number of subjects changes.

The program next uses any() to check whether at least one mark is below the passing threshold. If the condition is true, the status and grade are set to FAIL and F. Otherwise, an if/elif chain compares the percentage with the grade boundaries. Finally, a loop displays each subject and its mark, followed by the total, percentage, grade and status. This arrangement keeps the code compact while still showing the complete working process.

14. PYTHON SOURCE CODE

The following program is a complete console-based implementation of the algorithm described in the previous sections. It uses only Python's built-in functions and does not require the installation of additional packages. The code can be saved in a file named student_result_calculator.py and run with Python 3.

The program validates both the type and the range of the input. A non-numeric value is caught by the exception handler, while a number outside the range is rejected by the conditional statement. The grade-selection block is reached only after the program has checked whether every subject has been passed.

```
subjects = ["English", "Mathematics", "Computer Science", "Physics", "Chemistry"]
marks = []
PASS_MARK = 33
MAX_MARK = 100

for subject in subjects:
    while True:
        try:
            mark = float(input(f"Enter marks for {subject} (0-100): "))
            if mark < 0 or mark > MAX_MARK:
                print("Invalid marks. Enter a value from 0 to 100.")
                continue
            marks.append(mark)
            break
        except ValueError:
            print("Invalid input. Please enter a numeric mark.")

total = sum(marks)
maximum_total = len(subjects) * MAX_MARK
percentage = (total / maximum_total) * 100

if any(mark < PASS_MARK for mark in marks):
    status = "FAIL"
    grade = "F"
else:
    status = "PASS"
    if percentage >= 90:
    grade = "A+"
    elif percentage >= 80:
        grade = "A"
    elif percentage >= 70:
        grade = "B"
    elif percentage >= 60:
        grade = "C"
    elif percentage >= 45:
        grade = "D"
    else:
        grade = "E"

print("\n----- STUDENT RESULT -----")
for subject, mark in zip(subjects, marks):
    print(f"{subject}: {mark:g}/100")
print(f"Total: {total:g}/{maximum_total}")
print(f"Percentage: {percentage:.2f}%")
print(f"Grade: {grade}")
print(f"Status: {status}")
```

Explanation of the code

The subjects list stores the names that appear in the prompts and output. The loop requests a mark for each subject. The try/except block catches non-numeric input, while the range check rejects marks below zero or above 100. The accepted marks are stored in a list, and sum() calculates their total. The percentage formula compares that total with the maximum possible marks. Finally, any() checks whether one or more

subjects fall below the pass mark, and the if/elif chain selects the appropriate grade when the student passes.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Student Result Calculator</title>

  <!-- Connect CSS file -->
  <link rel="stylesheet" href="style.css">

  <!-- Connect JavaScript file -->
  <script src="script.js" defer></script>
</head>

<body>
  <main class="app-container">

    <header class="hero">
      <div class="logo">🎓</div>
      <p class="eyebrow">STUDENT MANAGEMENT SYSTEM</p>
      <h1>Student Result Calculator</h1>
      <p class="subtitle">
        Calculate marks, percentage and grades instantly.
      </p>
    </header>

    <section class="calculator-card">

      <div class="section-heading">
        <div>
          <h2>Student Details</h2>
          <p>Enter the student's information below.</p>
        </div>
        <span class="badge">ACADEMIC RESULT</span>
      </div>

      <form id="resultForm" novalidate>

        <div class="student-grid">
          <div class="field">
            <label for="name">Student Name</label>
            <input
              type="text"
              id="name"
              placeholder="Enter student name"
              value="Aditya"
              required
            >
          </div>

          <div class="field">
            <label for="roll">Roll Number</label>
            <input
```

```
        type="text"
        id="roll"
        placeholder="Enter roll number"
        value="23/Cse/25"
        required
      >
    </div>
  </div>

  <div class="marks-heading">
    <h2>Subject Marks</h2>
    <span>Maximum: 100 marks each</span>
  </div>

  <div class="marks-grid">
    <div class="field">
      <label for="english">📘 English</label>
      <input type="number" id="english"
          min="0" max="100" value="80" required>
    </div>

    <div class="field">
      <label for="math">📐 Mathematics</label>
      <input type="number" id="math"
          min="0" max="100" value="90" required>
    </div>

    <div class="field">
      <label for="computer">💻 Computer Science</label>
      <input type="number" id="computer"
          min="0" max="100" value="85" required>
    </div>

    <div class="field">
      <label for="physics">⚡ Physics</label>
      <input type="number" id="physics"
          min="0" max="100" value="75" required>
    </div>

    <div class="field full-width">
      <label for="chemistry">🧪 Chemistry</label>
      <input type="number" id="chemistry"
          min="0" max="100" value="70" required>
    </div>
  </div>

  <div class="button-row">
    <button type="submit" class="calculate-btn">
      Calculate Result <span>→</span>
    </button>

    <button type="reset" class="reset-btn">
      Reset
    </button>
```

```
  </div>

  <p class="rule-note">
    Passing rule: Minimum 33 marks in every subject.
  </p>

</form>

<div id="message" class="message" role="alert"
   aria-live="polite" hidden></div>

<section id="result" class="result-section" hidden
      aria-live="polite">
  <div class="result-top">
    <div>
      <p class="eyebrow">RESULT SUMMARY</p>
      <h2 id="resultName">Student Result</h2>
      <p id="resultRoll" class="result-roll"></p>
    </div>
    <span id="statusBadge" class="status-badge">PASS</span>
  </div>

  <div class="stats-grid">
    <div class="stat-box">
      <span>Total Marks</span>
      <strong id="total">0/500</strong>
    </div>

    <div class="stat-box">
      <span>Percentage</span>
      <strong id="percentage">0%</strong>
    </div>

    <div class="stat-box">
      <span>Grade</span>
      <strong id="grade">A</strong>
    </div>
  </div>

  <div class="result-table-wrap">
    <table>
      <thead>
        <tr>
          <th>Subject</th>
          <th>Marks</th>
          <th>Status</th>
        </tr>
      </thead>
      <tbody id="marksTable"></tbody>
    </table>
  </div>

  <button type="button" id="printBtn" class="print-btn">
    🖨️ Print Result
```

```
          </button>
        </section>

      </section>

      <footer>
        <p>Student Result Calculator · Mini Project</p>
        <p>Developed by Aditya | Computer Science & Engineering</p>
      </footer>

    </main>
</body>
</html>
```

Student Result Calculator — CSS Source Code

PDF version of theuploadedCSSfile.

```
@import
    url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;800&amp;
display=swap'
    );

:root {
    --bg: #0b1020;
    --card: #151d33;
    --input: #0f172a;
    --border: #2b3651;
    --text: #f1f5ff;
    --muted: #a4afc8;
    --purple: #8b5cf6;
    --blue: #4f8cff;
    --green: #34d399;
    --red: #fb7185;
}

* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

body {
    min-height: 100vh;
    padding: 35px 16px;
    font-family: 'Inter', Arial, sans-serif;
    color: var(--text);
    background:
        radial-gradient(ellipse at top left, #27205a 0%, transparent 40%),
        radial-gradient(ellipse at bottom right, #102f58 0%, transparent 40%),
        var(--bg);
}

.app-container {
width: 100%;
max-width: 850px;
margin: 0 auto;
}

.hero {
    text-align: center;
    margin-bottom: 30px;
}

.logo {
    width: 66px;
    height: 66px;
    margin: 0 auto 17px;
    display: grid;
    place-items: center;
    font-size: 30px;
    border-radius: 20px;
    background: linear-gradient(135deg, var(--purple), var(--blue));
    box-shadow: 0 10px 35px #6040c955;
}

.eyebrow {
    color:      #a99aff;
    font-size:     11px;
    font-weight:    800;
    letter-spacing: 2px;
    margin-bottom: 10px;
}

.hero h1 {
    font-size: clamp(27px, 5vw, 39px);
    line-height: 1.2;
    letter-spacing: -1px;
    margin-bottom: 12px;
}

.subtitle {
    color: var(--muted);
    font-size: 14px;
    line-height: 1.7;
}
```

```
.calculator-card {
padding: 30px;
background: linear-gradient(145deg, #19233c, #121a2e);
border: 1px solid #34405c;
border-radius: 24px;
box-shadow: 0 24px 70px #00000040;
}

.section-heading,
.result-top {
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 15px;
}

h2 {
    font-size: 19px;
}

.section-heading p,
.marks-heading span {
    color: var(--muted);
    font-size:     12px;
    margin-top:     7px;
    line-height: 1.6;
}

.badge {
    padding: 8px 10px;
    color: #c5b8ff;
    background: #8b5cf61c;
    border: 1px solid #8b5cf644;
    border-radius: 9px;
    font-size: 9px;
    font-weight: 800;
    letter-spacing: 0.6px;
    white-space: nowrap;
}

.student-grid,
.marks-grid {
    display: grid;
    grid-template-columns: repeat(2, minmax(0, 1fr));
    gap: 18px;
    margin-top: 24px;
}

.field {
    min-width: 0;
}

.field label {
    display:     block;
    color:     #dce5f8;
    font-size:    12px;
    font-weight:   600;
    margin-bottom: 9px;
}

.field input {
    display: block;
    width: 100%;
    min-width: 0;
    padding: 13px 14px;
    color: var(--text);
    font: inherit;
    font-size: 13px;
    background: var(--input);
    border: 1px solid var(--border);
    border-radius: 11px;
    outline: none;
    transition: border-color 0.2s, box-shadow 0.2s;
}

.field input::placeholder {
    color: #71809d;
}

.field input:focus {
    border-color: var(--purple);
    box-shadow: 0 0 0 3px #8b5cf625;
}
```

```
.marks-heading {
    display: flex;
    align-items: baseline;
    justify-content: space-between;
    gap: 10px;
    margin-top: 30px;
    padding-top: 24px;
    border-top: 1px solid var(--border);
}

.full-width {
    grid-column: 1 / -1;
}

.button-row {
    display: flex;
    gap: 12px;
    margin-top: 26px;
}

.calculate-btn,
.reset-btn,
.print-btn {
    padding: 14px 18px;
    font: inherit;
    font-size: 13px;
    font-weight: 700;
    border-radius: 11px;
    cursor: pointer;
    transition: transform 0.2s, filter 0.2s;
}

.calculate-btn:hover,
.reset-btn:hover,
.print-btn:hover {
    transform: translateY(-2px);
    filter: brightness(1.1);
}

.calculate-btn {
    flex: 1;
    display: flex;
    justify-content: center;
    gap: 12px;
    color: white;
    border: 0;
    background: linear-gradient(100deg, var(--purple), var(--blue));
    box-shadow: 0 8px 25px #6450e633;
}

.calculate-btn span {
    font-size: 18px;
    line-height: 14px;
}

.reset-btn {
    color: #d5def2;
    background: #202b43;
    border: 1px solid #3a4763;
}

.rule-note {
    color: var(--muted);
    text-align:  center;
    font-size:     11px;
    margin-top:    18px;
    line-height: 1.6;
}

.message {
    padding: 13px 15px;
    margin-top: 20px;
    color: #fecdd3;
    background: #88133733;
    border: 1px solid #fb718566;
    border-radius: 10px;
    font-size: 13px;
    line-height: 1.6;
}

.result-section {
    margin-top: 28px;
    padding-top: 25px;
    border-top: 1px solid var(--border);
```

```
}

.result-top h2 {
    margin-bottom: 6px;
    overflow-wrap: anywhere;
}

.result-roll {
    color: var(--muted);
    font-size: 12px;
}

.status-badge {
    padding: 9px 15px;
    border-radius: 10px;
    font-size: 12px;
    font-weight: 800;
    letter-spacing: 1px;
}

.status-badge.pass {
    color: #6ee7b7;
    background: #065f4638;
    border: 1px solid #34d39966;
}

.status-badge.fail {
    color: #fda4af;
    background: #88133738;
    border: 1px solid #fb718566;
}

.stats-grid {
    display: grid;
    grid-template-columns: repeat(3, minmax(0, 1fr));
    gap: 12px;
    margin: 22px 0;
}

.stat-box {
    padding: 17px 12px;
    background: linear-gradient(145deg, #202c48, #19233a);
    border: 1px solid #35415e;
    border-radius: 13px;
}

.stat-box span {
    display: block;
    color: var(--muted);
    font-size: 11px;
    margin-bottom: 9px;
}

.stat-box strong {
    font-size: clamp(17px, 3vw, 23px);
    overflow-wrap: anywhere;
}

.result-table-wrap {
    overflow-x: auto;
    border: 1px solid var(--border);
    border-radius: 12px;
}

table {
    width: 100%;
    border-collapse: collapse;
    font-size: 12px;
}

th,
td {
    padding: 13px 15px;
    text-align: left;
    border-bottom: 1px solid var(--border);
}

th {
    color: #c5b8ff;
    background: #202943;
    font-size: 11px;
}

td {
```

```
    color: #dce5f8;
}

tbody tr:last-child td {
    border-bottom: 0;
}

.subject-pass {
    color: var(--green);
    font-weight: 700;
}

.subject-fail {
    color: var(--red);
    font-weight: 700;
}

.print-btn {
    width: 100%;
    margin-top: 18px;
    color: #e8eaff;
    background: #293652;
    border: 1px solid #435170;
}

footer {
    padding: 25px 10px 5px;
    color: #8e9bb6;
    text-align: center;
    font-size: 11px;
    line-height: 1.9;
}

[hidden] {
    display: none !important;
}

@media(max-width: 600px) {
    body {
        padding: 22px 12px;
    }

    .calculator-card {
        padding: 20px 16px;
        border-radius: 18px;
    }

    .section-heading {
        align-items: flex-start;
        flex-direction: column;
    }

    .student-grid,
    .marks-grid {
        grid-template-columns: 1fr;
        gap: 16px;
    }

    .full-width {
        grid-column: auto;
    }

    .marks-heading {
        align-items: flex-start;
        flex-direction: column;
    }

    .stats-grid {
        gap: 7px;
    }

    .stat-box {
        padding: 13px 9px;
    }

    .stat-box span {
        font-size: 10px;
    }

    .button-row {
        flex-direction: column;
    }

    .reset-btn {
```

```
        width: 100%;
    }

    th,
    td{
        padding: 11px 9px;
    }
}

@mediaprint {
    body {
        padding: 0;
        color: #111;
        background: white;
    }

    .hero,
    form,
    footer,
    .print-btn,
    .message {
        display: none !important;
    }

    .app-container {
        max-width: 100%;
    }

    .calculator-card {
        padding: 0;
        color: #111;
        background: white;
        border: 0;
        box-shadow: none;
    }

    .result-section {
        display: block !important;
    }

    .result-section h2,
    .result-section p,
    .stat-box strong,
    td{
        color: #111;
    }

    .stat-box,
    th{
        color: #111;
        background: #f0f0f0;
    }
}
```

```
const form = document.getElementById("resultForm");
const resultSection = document.getElementById("result");
const messageBox = document.getElementById("message");

const subjects = [
  { name: "English", id: "english" },
  { name: "Mathematics", id: "math" },
  { name: "Computer Science", id: "computer" },
  { name: "Physics", id: "physics" },
  { name: "Chemistry", id: "chemistry" }
];

// Display an error message
function showMessage(message) {
  messageBox.textContent = message;
  messageBox.hidden = false;
  resultSection.hidden = true;
}

// Hide the error message
function hideMessage() {
  messageBox.textContent = "";
  messageBox.hidden = true;
}

// Calculate the grade
function calculateGrade(percentage, passed) {
  if (!passed) return "F";
  if (percentage >= 90) return "A+";
  if (percentage >= 80) return "A";
  if (percentage >= 70) return "B";
  if (percentage >= 60) return "C";
  if (percentage >= 45) return "D";
  return "E";
}

// Handle form submission
form.addEventListener("submit", function (event) {
  event.preventDefault();
  hideMessage();

  const name = document.getElementById("name").value.trim();
  const roll = document.getElementById("roll").value.trim();

  // Check student details
  if (name === "" || roll === "") {
    showMessage("Please enter the student name and roll number.");
    return;
  }

  const marks = [];

  // Validate all five subject marks
```

```
for (const subject of subjects) {
  const input = document.getElementById(subject.id);
  const value = input.valueAsNumber;

  if (
    input.value.trim() === "" ||
    !Number.isFinite(value) ||
    value < 0 ||
    value > 100
  ) {
    showMessage(
      "Please enter valid marks from 0 to 100 in every subject."
    );
    input.focus();
    return;
  }

  marks.push(value);
}

// Calculate total and percentage
const total = marks.reduce((sum, mark) => sum + mark, 0);
const percentage = (total / 500) * 100;

// A student must score at least 33 in every subject
const passed = marks.every(mark => mark >= 33);

const grade = calculateGrade(percentage, passed);
const finalResult = passed ? "PASS" : "FAIL";

// Display student details
document.getElementById("resultName").textContent = name;
document.getElementById("resultRoll").textContent =
  "Roll Number: " + roll;

document.getElementById("total").textContent =
  total + " / 500";

document.getElementById("percentage").textContent =
  percentage.toFixed(2) + "%";

document.getElementById("grade").textContent = grade;

// Update PASS / FAIL badge
const statusBadge = document.getElementById("statusBadge");

statusBadge.textContent = finalResult;
statusBadge.className =
  "status-badge " + (passed ? "pass" : "fail");

// Create subject-wise result rows
const marksTable = document.getElementById("marksTable");
marksTable.replaceChildren();
```

```
  subjects.forEach((subject, index) => {
    const mark = marks[index];
    const subjectPassed = mark >= 33;

    const row = document.createElement("tr");

    const nameCell = document.createElement("td");
    nameCell.textContent = subject.name;

    const marksCell = document.createElement("td");
    marksCell.textContent = mark + " / 100";

    const statusCell = document.createElement("td");
    statusCell.textContent = subjectPassed ? "PASS" : "FAIL";
    statusCell.className =
      subjectPassed ? "subject-pass" : "subject-fail";

    row.append(nameCell, marksCell, statusCell);
    marksTable.appendChild(row);
  });

  // Show the result
  resultSection.hidden = false;
  resultSection.scrollIntoView({
    behavior: "smooth",
    block: "nearest"
  });
});

// Reset the displayed result when the form is reset
form.addEventListener("reset", function () {
  hideMessage();
  resultSection.hidden = true;
});

// Print the result
document.getElementById("printBtn").addEventListener("click", function () {
  window.print();
});
```

SAMPLE EXECUTION AND OUTPUT

The following example uses the sample marks included in this report. The program asks for each subject one at a time and then prints a summary.

```
Enter marks for English (0-100): 80
Enter marks for Mathematics (0-100): 90
Enter marks for Computer Science (0-100): 85
Enter marks for Physics (0-100): 75
Enter marks for Chemistry (0-100): 70

----- STUDENT RESULT -----
English: 80/100
Mathematics: 90/100
Computer Science: 85/100
Physics: 75/100
Chemistry: 70/100
Total: 400/500
Percentage: 80.00%
Grade: A
Status: PASS
```

How to run the program

1. Install Python 3 if it is not already installed.
2. Open a text editor or Python IDE and paste the source code.
3. Save the file as ***student_result_calculator.py***.
4. Open a terminal or command prompt in the folder containing the file.
5. Run `python student_result_calculator.py` (on some systems, use `python3 student_result_calculator.py`).
6. Enter a numeric mark from 0 to 100 for each subject when prompted.

Note: The output above is an illustrative sample run. Actual results change according to the marks entered by the user.

Student Result Calculator

Calculate marks, percentage and grades instantly.

Mini Project

- Home
- About
- Help

"Small steps in learning today, big dreams tomorrow."



Student Details

Enter the student's information below.

Student Name: Aditya

Roll Number: 23/Cse/25

Subject Marks

Maximum: 100 marks each

English	Mathematics	Computer Science
80	90	85

Physics	Chemistry
75	70

Calculate Result →

Reset

Passing rule: Minimum 33 marks in every subject.

Student Result Calculator

Calculate marks, percentage and grades instantly.

Mini Project



Better Education Brighter Future

"Small steps in learning today, big dreams tomorrow."

- Easy to Use
- Accurate Calculation
- Instant Result

Student Details

Enter the student's information below.

Student Name: Aditya

Roll Number: 23/Cse/25

Subject Marks

Maximum: 100 marks each

English	Mathematics	Computer Science
80	90	85

Physics	Chemistry
75	70

Calculate Result →

Reset

Passing rule: Minimum 33 marks in every subject.

Result Calculated Successfully!

RESULT SUMMARY

Aditya

Roll Number: 23/Cse/25

PASS

Total Marks	Percentage	Grade
400 / 500	80.00%	A

Subject-wise Marks

Subject	Marks	Status
English	80 / 100	PASS
Mathematics	90 / 100	PASS
Computer Science	85 / 100	PASS
Physics	75 / 100	PASS
Chemistry	70 / 100	PASS

Print Result

15. INPUT AND OUTPUT EXAMPLES

A sample run helps verify that the program is performing the calculations correctly. Suppose the user enters 80 in English, 90 in Mathematics, 85 in Computer Science, 75 in Physics and 70 in Chemistry. The total is 80 + 90 + 85 + 75 + 70 = 400. Since the maximum aggregate is 500, the percentage is (400 / 500) × 100 = 80 percent.

Every subject mark in this example is at least 33, so the student passes. The percentage of 80 meets the threshold for grade A. The output should therefore show the five subject marks, total 400/500, percentage 80.00%, grade A and status PASS. This example can be used during a demonstration because the arithmetic is easy to verify manually.

A second example checks the fail condition. If the marks are 95, 92, 88, 30 and 90, one subject is below the minimum passing mark. The program must display FAIL and grade F even though the total and percentage are high. This confirms that the program checks the subject-wise condition before assigning a passing grade.

Invalid entries should also be tested. If the user enters the word 'abc', the program should display a message requesting a numeric value. If the user enters 105, it should explain that the mark must be between 0 and 100 and ask again. These cases show that the program is designed to handle mistakes in data entry instead of crashing or producing a misleading result.

16. TESTING AND VALIDATION

Testing is an important part of software development because a program that runs without an error is not necessarily a program that produces the correct answer. Testing compares the actual output with the expected output for carefully selected inputs. For the Student Result Calculator, the tests should cover ordinary marks, grade boundaries, pass/fail conditions and invalid entries.

A normal test uses marks such as 80, 90, 85, 75 and 70. The expected total is 400, the percentage is 80, the grade is A and the status is PASS. A subject-failure test uses at least one mark below 33 and checks that the result is FAIL even when the overall percentage is high.

Boundary testing focuses on values near the limits. Marks of 32 and 33 are especially important because they fall on opposite sides of the subject passing threshold. Grade-boundary tests should check percentages at and around 45, 60, 70, 80 and 90 to confirm that the if/elif conditions assign the intended grade.

Invalid-input testing includes text instead of a number, a negative mark and a mark above 100. The expected behaviour is that the program displays a helpful message and asks for a corrected value. A simple test table can record the input, expected result, actual result and whether the test passed. Testing should be repeated after any change to the code.

17. ADVANTAGES OF THE SYSTEM

One major advantage is that the program performs repeated calculations automatically. The user does not need to add five marks manually or calculate the percentage separately. This saves a small amount of time for one result and can save more time if the program is later extended to process multiple students.

The program applies the same rules to every valid set of marks. Consistency is important because two students with the same marks should receive the same total, percentage and grade. Since the rules are written in code, they can be reviewed and tested rather than relying on memory during each calculation.

Input validation is another advantage. It prevents a negative mark or a mark greater than 100 from being accepted accidentally. Exception handling also helps the program recover when a user enters text instead of a number. Clear output makes it easier to check the result and explain the calculation during a practical demonstration.

The project is also useful from an educational perspective. It brings together several basic programming ideas in a single application and encourages the student to think about program structure, decision order and test cases. The code is short enough to understand but can be improved in many ways as the learner gains experience.

18. LIMITATIONS AND FUTURE ENHANCEMENTS

The current version has a deliberately limited scope. It calculates the result for one student at a time, uses five fixed subjects and does not preserve the result after the program closes. The user must enter all marks manually. There is no database, student search facility or automatic import of marks from another system.

The console interface is suitable for learning but may not be ideal for users who are unfamiliar with command-line programs. The program also assumes that all subjects have the same maximum mark. If a course uses different maximum marks or weighted assessments, the formula and data structure would need to be updated.

Several enhancements are possible. The program could ask for a student's name and roll number, allow the user to choose subjects, process a list of students and save results in a CSV file. A database could be added for persistent storage, while a graphical interface could make data entry more convenient. Automated unit tests could be written for the grade and pass/fail functions.

A production-quality result system would require additional safeguards, including authentication, permissions, backups, change history and an institution-approved grading policy. These features are not included in the micro-project because its primary purpose is to demonstrate the fundamentals of Python programming.

19. SECURITY, PRIVACY AND DATA ACCURACY

Marks are educational information, so they should be handled responsibly. In this demonstration, the program runs locally and does not send marks to an online service. This keeps the implementation simple, but it does not remove the need to think carefully about data accuracy and privacy when the project is expanded.

The program can check whether a mark is numeric and within the permitted range, but it cannot know whether the entered value is the official mark awarded by a teacher. The accuracy of the output therefore depends on the accuracy of the input. In a real system, marks may need to be entered by authorised staff and checked against official records.

If file storage or online access is added, the system should limit who can view or change records. Stored files should be protected from accidental sharing, and backups should be handled securely. It is also useful to keep a record of changes so that mistakes can be traced and corrected.

These considerations illustrate a general principle of software development: correct calculations are only one part of a reliable information system. The system must also handle data responsibly and make its limitations clear to the people using it.

20. RESULT, CONCLUSION AND REFERENCES

Result of the project

The Student Result Calculator successfully demonstrates the calculation of total marks, percentage, grade and pass/fail status using the specified rules. With marks of 80, 90, 85, 75 and 70, the program calculates 400 out of 500 and a percentage of 80.00. Since every subject mark is at least 33, it assigns grade A and displays PASS.

Conclusion

This micro-project applies fundamental Python concepts to a practical academic problem. The work involves understanding the requirements, preparing an algorithm, validating input, implementing calculations and testing the output. The project demonstrates that a correct result depends on both accurate arithmetic and correct decision-making. In particular, the program checks the minimum passing mark in each subject before assigning a passing grade.

The project meets its educational objective by providing a small, understandable application that can be executed in a standard Python environment. Its limitations also provide a clear path for future work, such as support for multiple students, persistent storage and a graphical interface. If the application is ever used beyond a classroom demonstration, the grading rules should be verified and student data should be protected appropriately.

References

1. Python Software Foundation, Python 3 Documentation: https://docs.python.org/3/
2. Python Software Foundation, Built-in Functions: https://docs.python.org/3/library/functions.html
3. Python Software Foundation, Control Flow Tools: https://docs.python.org/3/tutorial/controlflow.html
4. Python Software Foundation, Errors and Exceptions: https://docs.python.org/3/tutorial/errors.html
5. Project-specific grading thresholds and sample marks defined for this educational micro-project.

— End of Report —

APPENDIX A: COMPLETE PYTHON SOURCE CODE

This appendix provides the full runnable source code for the Student Result Calculator. Save it as ***student_result_calculator.py*** and run it using Python 3. The program accepts marks for five subjects, validates the input, calculates the total and percentage, and displays the grade and result status.

Source code

```
subjects = ["English", "Mathematics", "Computer Science", "Physics", "Chemistry"]
marks = []
PASS_MARK = 33
MAX_MARK = 100

for subject in subjects:
    while True:
        try:
            mark = float(input(f"Enter marks for {subject} (0-100): "))
            if mark < 0 or mark > MAX_MARK:
                print("Invalid marks. Enter a value from 0 to 100.")
                continue
            marks.append(mark)
            break
        except ValueError:
            print("Invalid input. Please enter a numeric mark.")

total = sum(marks)
maximum_total = len(subjects) * MAX_MARK
percentage = (total / maximum_total) * 100

if any(mark < PASS_MARK for mark in marks):
    status = "FAIL"
    grade = "F"
else:
    status = "PASS"
    if percentage >= 90:
    grade = "A+"
    elif percentage >= 80:
        grade = "A"
    elif percentage >= 70:
        grade = "B"
    elif percentage >= 60:
        grade = "C"
    elif percentage >= 45:
        grade = "D"
    else:
        grade = "E"

print("\n----- STUDENT RESULT -----")
for subject, mark in zip(subjects, marks):
    print(f"{subject}: {mark:g}/100")
print(f"Total: {total:g}/{maximum_total}")
print(f"Percentage: {percentage:.2f}%")
print(f"Grade: {grade}")
print(f"Status: {status}")
```

Important rules implemented: Each subject is out of 100 marks. A student must score at least 33 in every subject to pass. If any subject is below 33, the status is FAIL and the grade is F. Otherwise, the overall percentage determines the grade: A+ (90% and above), A (80–89.99%), B (70–79.99%), C (60–69.99%), D (45–59.99%), and E (below 45%). Non-numeric input and marks outside 0–100 are rejected.

SAMPLE EXECUTION AND OUTPUT

The following example uses the sample marks included in this report. The program asks for each subject one at a time and then prints a summary.

```
Enter marks for English (0-100): 80
Enter marks for Mathematics (0-100): 90
Enter marks for Computer Science (0-100): 85
Enter marks for Physics (0-100): 75
Enter marks for Chemistry (0-100): 70

----- STUDENT RESULT -----
English: 80/100
Mathematics: 90/100
Computer Science: 85/100
Physics: 75/100
Chemistry: 70/100
Total: 400/500
Percentage: 80.00%
Grade: A
Status: PASS
```

How to run the program

1. Install Python 3 if it is not already installed.
2. Open a text editor or Python IDE and paste the source code.
3. Save the file as ***student_result_calculator.py***.
4. Open a terminal or command prompt in the folder containing the file.
5. Run `python student_result_calculator.py` (on some systems, use `python3 student_result_calculator.py`).
6. Enter a numeric mark from 0 to 100 for each subject when prompted.

Note: The output above is an illustrative sample run. Actual results change according to the marks entered by the user.